

PONTIFÍCIA UNIVERSIDADE CATÓLICA DE CAMPINAS
Escola Politécnica- Faculdade de Análise de Sistemas
Engenharia de Software

BRUNO REITANO FIGUEROLA
GABRIEL FLORES BONATTO
HENRY GABRIEL PIOZZI
PEDRO XIMENES COSTA
ROGÉRIO MEDINA

GRUPO 02

EFC 1: REFATORAÇÃO GUIADA POR SOLID, CLEAN CODE E PADRÕES GoF

PADRÕES E ARQUITETURA DE SOFTWARE

CAMPINAS
1º Semestre 2026

Sumário

1	IDENTIFICAÇÃO DAS VIOLAÇÕES SOLID	4
1.1	SRP – Single Responsibility Principle	4
1.2	OCP – Open/Closed Principle	5
1.3	LSP – Liskov Substitution Principle	6
1.4	ISP – Interface Segregation Principle	7
1.5	DIP – Dependency Inversion Principle	8
2	SOLUÇÕES IMPLEMENTADAS	10
2.1	SRP – Extração em camadas	10
2.2	OCP – Padrão Strategy	11
2.3	LSP – Documentação e não-reprodução	12
2.4	ISP – Interfaces abstratas por camada	12
2.5	DIP – Injeção de dependência explícita	13
3	DIAGRAMA UML DE CLASSES	15
4	MELHORIAS DE CLEAN CODE	16
5	EXTENSÕES IMPLEMENTADAS	18
5.1	Extensão 1 – Pagamento em Criptomoeda	18
5.2	Extensão 2 – Notificação WhatsApp	18
5.3	Extensão 3 – Desconto Progressivo por Volume	19
6	SAÍDA DOS TESTES E RELATÓRIO DE MÉTRICAS	21
6.1	Testes Automatizados	21
6.2	Cobertura de Código	21
6.3	Lint — PEP 8 / Boas Práticas (ruff)	22
6.4	Checagem de Tipos Estáticos (mypy --strict)	22
6.5	Complexidade Ciclomática (radon cc)	22
6.6	Resumo das Métricas	22
7	REFLEXÃO METACOGNITIVA	23

7.1	O princípio que ainda não dominei	23
7.2	Como usei IA neste trabalho	23

1 IDENTIFICAÇÃO DAS VIOLAÇÕES SOLID

1.1 SRP – Single Responsibility Principle

Violação 1 – Classe Sis com múltiplas responsabilidades

Arquivo: legacy.py, linhas 6–160

```
class Sis:
    def add_ped(self, n, its, t): ... # cria pedido + calcula total + notifica
    def upd_st(self, id, s): ... # atualiza status + notifica + pontua
    def proc_pag(self, id, m, vl): ... # processa pagamento
    def gerar_rel(self, tipo): ... # gera relatorios
    def validar_estoque(self, its): ... # valida estoque
```

A classe Sis acumula persistência, cálculo de totais, notificações, pagamentos, controle de estoque e relatórios. Qualquer mudança em uma dessas responsabilidades obriga a mexer na mesma classe, aumentando o risco de regressão. Se o formato do relatório mudar, a lógica de pedidos está no mesmo arquivo e pode ser acidentalmente afetada.

Impacto prático: adicionar um novo canal de notificação exigia modificar o mesmo método que calcula desconto VIP.

Violação 2 – Método add_ped com três responsabilidades

Arquivo: legacy.py, linhas 15–49

```
def add_ped(self, n, its, t):
    # 1) calcula total com if/elif por tipo de item
    tot = 0
    for i in its:
        if i['tipo'] == 'normal': tot += i['p'] * i['q']
        elif i['tipo'] == 'desc10': tot += i['p'] * i['q'] * 0.9
    ...
```

```
# 2) aplica desconto por tipo de cliente
if t == 'vip': tot = tot * 0.95
# 3) persiste no SQLite
self.c.execute("INSERT INTO ped ...")
# 4) envia notificacao
if t == 'normal': print(f"Email enviado para {n}...")
```

Um único método realiza cálculo de desconto, persistência e notificação. Testar qualquer uma dessas partes isoladamente é impossível sem instanciar o banco de dados.

Impacto prático: impossível testar a lógica de desconto sem criar um banco SQLite real.

1.2 OCP – Open/Closed Principle

Violação 1 – Descontos hardcoded em `add_ped`

Arquivo: `legacy.py`, linhas 18–31

```
for i in its:
    if i['tipo'] == 'normal':
        tot += i['p'] * i['q']
    elif i['tipo'] == 'desc10':
        tot += i['p'] * i['q'] * 0.9
    elif i['tipo'] == 'desc20':
        tot += i['p'] * i['q'] * 0.8
    elif i['tipo'] == 'frete_gratis':
        tot += i['p'] * i['q']
```

Cada novo tipo de desconto exige modificar diretamente o método. O código não está fechado para modificação; qualquer extensão é uma edição no arquivo original.

Impacto prático: implementar "desconto por volume" forçaria editar `add_ped`, arriscando quebrar os descontos já existentes.

Violação 2 – Métodos de pagamento hardcoded em proc_pag

Arquivo: legacy.py, linhas 116–139

```
if m == 'cartao':
    print("Processando pagamento com cartao...")
    self.upd_st(id, 'aprovado')
    return True
elif m == 'pix':
    print("Gerando QR Code PIX...")
    self.upd_st(id, 'aprovado')
    return True
elif m == 'boleto':
    print("Gerando boleto...")
    return True
```

Adicionar pagamento em criptomoeda exigiria um novo elif neste método, abrindo o código para modificação.

Impacto prático: integrar uma gateway de pagamento externa forçaria modificar código já testado em produção.

1.3 LSP – Liskov Substitution Principle

Violação 1 – PedEspecial.upd_st ignora comportamento do pai

Arquivo: legacy.py, linhas 183–190

```
class PedEspecial(Sis):
    def upd_st(self, id, s):
        # PedEspecial pula direto para qualquer estado
        # ignorando transicoes intermediarias do pai
        p = self.get_ped(id)
        if p:
            self.c.execute("UPDATE ped SET st=? WHERE id=?", (s, id))
            self.db.commit()
            print(f"Pedido especial {id} -> {s}")
```

Sis.upd_st envia notificações por e-mail/SMS e concede pontos de fidelidade ao entregar. PedEspecial.upd_st ignora tudo isso: trocar um Sis por PedEspecial altera silenciosamente o comportamento observável. Clientes VIP perderiam seus pontos sem aviso.

Impacto prático: qualquer código que receba um Sis e chame upd_st se comporta diferente com PedEspecial sem lançar exceção: violação silenciosa.

Violação 2 – PedEspecial.add_ped quebra o contrato de tipos

Arquivo: legacy.py, linhas 164–174

```
def add_ped(self, n, its, t):
    for i in its:
        if i['tipo'] == 'normal': tot += i['p'] * i['q']
        elif i['tipo'] == 'desc10': tot += i['p'] * i['q'] * 0.9
        elif i['tipo'] == 'desc20': tot += i['p'] * i['q'] * 0.8
        # 'frete_gratis' nao tratado: silenciosamente ignorado
    tot = tot * 1.15
```

O pai Sis aceita itens do tipo frete_gratis. PedEspecial silenciosamente não reconhece esse tipo, totalizando zero para esses itens. O subtype não honra o contrato do supertipo.

Impacto prático: pedidos com frete_gratis em PedEspecial resultam em valores incorretos sem exceção.

1.4 ISP – Interface Segregation Principle

Violação 1 – Ausência total de interfaces

Arquivo: legacy.py, classe Sis

Não existem interfaces abstratas no sistema legado. Todo cliente que precisar de qualquer funcionalidade deve depender da classe Sis inteira: incluindo métodos de relatório, estoque, pagamento e notificação, mesmo que precise apenas de um deles.

Impacto prático: um módulo de relatório que só precisa de gerar_rel fica acoplado a proc_pag, validar_estoque e toda a lógica de banco.

Violação 2 – Interface "gorda" implícita em Sis

Arquivo: legacy.py, linhas 51–160

```
class Sis:
    def get_ped(self, id): ...
    def upd_st(self, id, s): ...
    def calc_tot_cli(self, n): ...
    def gerar_rel(self, tipo): ...
    def proc_pag(self, id, m, vl): ...
    def validar_estoque(self, its): ...
    def cancelar_pedido(self, id): ...
```

Sete métodos públicos com responsabilidades completamente distintas em uma única interface implícita. ISP pede interfaces coesas e mínimas.

Impacto prático: qualquer teste que mocke Sis precisa implementar todos os métodos, mesmo os irrelevantes para o teste.

1.5 DIP – Dependency Inversion Principle

Violação 1 – Acoplamento direto ao SQLite

Arquivo: legacy.py, linhas 7–9

```
def __init__(self):
    self.db = sqlite3.connect('loja.db') # concreto, nao abstrato
    self.c = self.db.cursor()
```

Sis cria sua própria dependência de banco de dados. Impossível testar sem arquivo físico loja.db ou trocar por outro banco sem modificar a classe.

Impacto prático: testes sempre criam arquivos reais no disco; migrar para PostgreSQL exigiria reescrever Sis.

Violação 2 – Lógica de notificação concreta embutida

Arquivo: legacy.py, linhas 40–47

```
if t == 'normal':  
    print(f"Email enviado para {n}: Pedido recebido!")  
elif t == 'vip':  
    print(f"Email enviado para {n}: Pedido recebido!")  
    print(f"SMS enviado para {n}: Pedido VIP recebido!")  
elif t == 'corporativo':  
    print(f"Email enviado para {n}: Pedido recebido!")  
    print(f"Notificacao enviada ao gerente de conta de {n}")
```

A regra de negócio de pedidos depende diretamente da implementação de notificação. Não há abstração: a lógica de "quem notifica" está embutida na lógica de "como criar pedido".

Impacto prático: integrar e-mail real (SMTP) exigiria modificar o método de criação de pedidos.

2 SOLUÇÕES IMPLEMENTADAS

2.1 SRP – Extração em camadas

A classe Sis foi desmembrada em seis classes com responsabilidade única, organizadas em camadas explícitas:

Classe	Responsabilidade
OrderRepository	persistência SQLite
OrderService	orquestração de pedidos
PaymentService	processamento de pagamentos
ReportService	geração de relatórios
StockService	validação de estoque
ObserverNotificationService	despacho de notificações

Padrão GoF: Repository (isolamento da persistência).

Intenção (GoF): Separa a lógica de negócio do mecanismo de acesso a dados, fornecendo uma interface orientada a coleções para acesso a objetos de domínio.

Diagrama parcial:

«ABC»

IOrderRepository

+ save(order: Order) → int

+ find_by_id(id: int) → Optional[Order]

+ find_by_client(client: str) → List[Order]



OrderRepository (implementação SQLite)

Antes:

```
class Sis:
```

```
    def add_ped(self, n, its, t):
```

```
        # calcula, persiste e notifica no mesmo método
```

Depois:

```
class OrderService:
```

```
    def __init__(self, repository: IOrderRepository,
                  notification_service: INotificationService) -> None:
        self._repository = repository
        self._notification = notification_service
```

```
    def create_order(self, cliente: str, itens_raw: List[dict], tipo: str) -> int:
        itens = [OrderItem.from_dict(i) for i in itens_raw]
        total = self._calculate_total(itens, tipo)
        order = Order(cliente=cliente, itens=itens, tipo=tipo, total=total)
        order_id = self._repository.save(order)
        self._notification.notify_order_created(cliente, tipo)
        return order_id
```

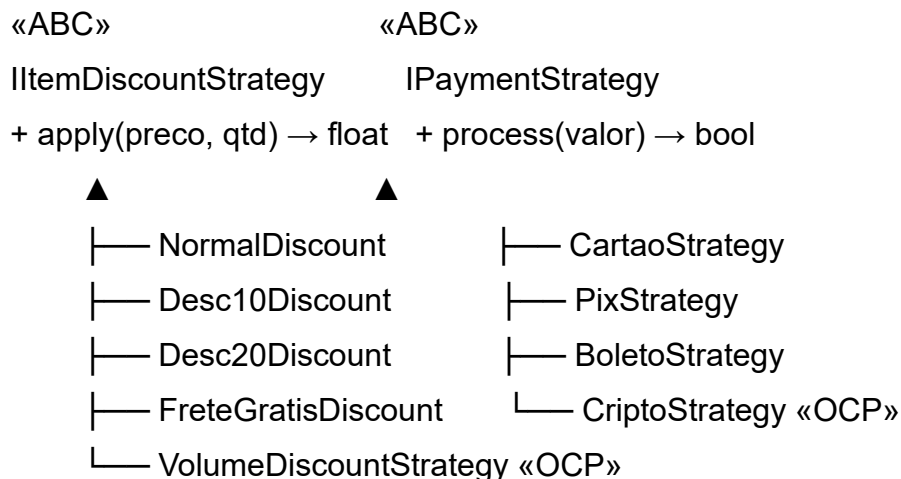
2.2 OCP – Padrão Strategy

Algoritmos de desconto e pagamento extraídos para hierarquias de Strategy. Novas variantes são adicionadas criando novas classes, sem tocar nas existentes.

Padrão GoF: Strategy.

Intenção (GoF): Define uma família de algoritmos, encapsula cada um e os torna intercambiáveis. Permite que o algoritmo varie independentemente dos clientes que o utilizam.

Diagrama parcial:



Classes criadas: `ItemDiscountStrategy`, `NormalDiscount`, `Desc10Discount`, `Desc20Discount`, `FreteGratisDiscount`, `IPaymentStrategy`, `CartaoStrategy`, `PixStrategy`, `BoletoStrategy`.

Antes:

```
if i['tipo'] == 'desc10':
    tot += i['p'] * i['q'] * 0.9
```

Depois:

```
class Desc10Discount(ItemDiscountStrategy):
    def apply(self, preco: float, quantidade: int) -> float:
        return preco * quantidade * 0.9

# Em OrderService._calculate_total:
strategy = ITEM_DISCOUNT_REGISTRY.get(item.tipo, NormalDiscount())
total += strategy.apply(item.preco, item.quantidade)
```

2.3 LSP – Documentação e não-reprodução

`PedEspecial` permanece no `legacy.py` intocável (requisito do enunciado). O código novo em `src/` não reproduz a hierarquia problemática. A violação é documentada aqui como evidência de que foi identificada. O `src/main.py` não instancia `PedEspecial`.

A violação consiste em `PedEspecial.upd_st` ignorar notificações e pontos de fidelidade do pai, tornando os dois tipos não substituíveis. Em Liskov, `Sis s = new PedEspecial()` deveria se comportar como `Sis`: mas não se comporta.

2.4 ISP – Interfaces abstratas por camada

Criadas três interfaces ABCs, uma por fronteira de colaboração externa.

Classes criadas: `IOrderRepository`, `INotificationService`, `IPaymentStrategy`.

Antes: sem interfaces, dependência direta na classe concreta.

Depois:

```

class IOrderRepository(ABC):
    @abstractmethod
    def save(self, order: Order) -> int: ...
    @abstractmethod
    def find_by_id(self, order_id: int) -> Optional[Order]: ...

```

```

class OrderRepository(IOrderRepository):
    ... # implementacao SQLite isolada aqui

```

2.5 DIP – Injeção de dependência explícita

Todos os serviços recebem suas dependências via construtor, tipadas como abstrações.

Padrão GoF: Observer/Pub-Sub (notificações), Factory Method (criação de pedidos).

Observer — Intenção (GoF): Define uma dependência um-para-muitos entre objetos, de modo que quando um objeto muda de estado, todos os seus dependentes são notificados automaticamente.

Diagrama parcial Observer:

```

NotificationPublisher
+ subscribe(obs: INotificationObserver) → None
+ publish(cliente, tipo, evento) → None
  | notifica
  ▼
«ABC»
INotificationObserver
+ notify(cliente, tipo_cliente, evento) → None
  ▲
  ├── EmailObserver
  ├── SmsObserver
  ├── AccountManagerObserver
  └── WhatsAppObserver «OCP»

```

Factory Method — Intenção (GoF): Define uma interface para criar um objeto, mas deixa as subclasses decidirem qual classe instanciar. Permite adiar para subclasses a decisão sobre qual objeto criar.

Diagrama parcial Factory:

«ABC»

IOrderFactory

+ create(cliente, itens, total) → Order



└─ NormalOrderFactory

└─ VipOrderFactory

└─ CorporativoOrderFactory

OrderService recebe IOrderFactory via construtor (DIP)

Antes:

class Sis:

def __init__(self):

self.db = sqlite3.connect('loja.db') # cria o concreto

Depois:

class OrderService:

def __init__(

self,

repository: IOrderRepository,

notification_service: INotificationService,

) -> None:

self._repository = repository

self._notification = notification_service

Em main.py: unico ponto de composicao:

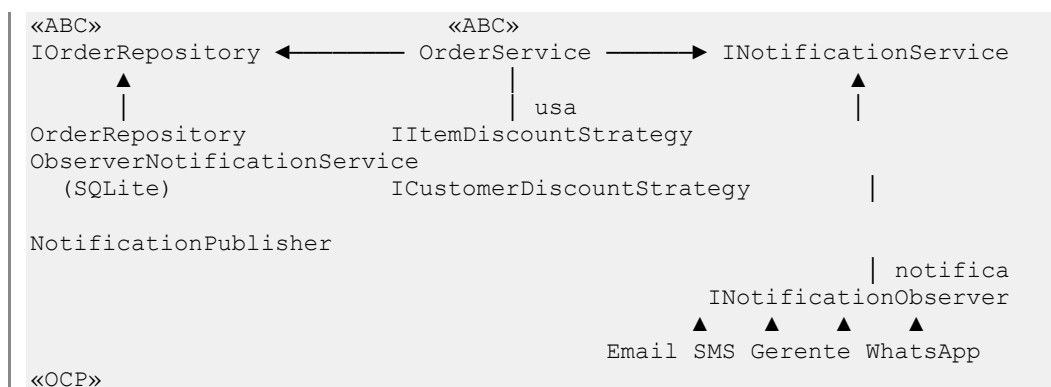
repo = OrderRepository()

notification = ObserverNotificationService()

order_service = OrderService(repo, notification)

3 DIAGRAMA UML DE CLASSES

O diagrama completo está em docs/diagrama.puml (PlantUML). Reprodução parcial abaixo mostrando as dependências centrais respeitando DIP:



Camadas: models → repositories → services → strategies / observers / factories.

Setas de dependência apontam sempre para abstrações (interfaces ABCs), nunca para concretos.

4 MELHORIAS DE CLEAN CODE

Nomenclatura

Legado	Refatorado	Motivo
Sis	OrderService, PaymentService, ...	nome revela intenção
add_ped	create_order	verbo + substantivo claro
upd_st	update_status	sem abreviação
proc_pag	process	contexto dado pela classe
gerar_rel	sales_report / customers_report	métodos específicos
n, its, t	cliente, itens, tipo	parâmetros sem abreviação
tot, tot_g	total, grand_total	variáveis legíveis

Métodos extraídos

- OrderService._calculate_total: lógica de desconto extraída do create_order
- OrderService._award_points: concessão de pontos extraída do update_status
- NotificationPublisher.publish: loop de observers extraído para método próprio.

Abstrações criadas

- ItemDiscountStrategy e ICustomerDiscountStrategy: eliminaram os if/elif de desconto
- IPaymentStrategy: eliminou os if/elif de método de pagamento
- INotificationObserver: eliminou os if/elif de canal de notificação
- IOrderFactory: padronizou criação de pedidos por tipo de cliente

Eliminação de duplicação

- Lógica de desconto por item estava duplicada em Sis.add_ped e PedEspecial.add_ped: consolidada em ITEM_DISCOUNT_REGISTRY
- Lógica de notificação estava duplicada em add_ped e upd_st: centralizada em ObserverNotificationService

5 EXTENSÕES IMPLEMENTADAS

As três extensões provam OCP empiricamente: nenhuma classe existente foi modificada.

5.1 Extensão 1 – Pagamento em Criptomoeda

Arquivo adicionado: src/strategies/crypto_payment.py

Arquivos modificados: nenhum

```
from src.strategies.payment_strategy import IPaymentStrategy
```

```
class CriptoStrategy(IPaymentStrategy):
```

```
    FEE = 0.02
```

```
    def process(self, valor: float) -> bool:
```

```
        taxa = valor * self.FEE
```

```
        print(f"Pagamento cripto: valor={valor:.2f}, taxa={taxa:.2f}")
```

```
        print("Transacao blockchain confirmada!")
```

```
        return True
```

```
    def approves_order(self) -> bool:
```

```
        return True
```

Por que foi simples: PaymentService depende de IPaymentStrategy, não de classes concretas. Basta registrar CriptoStrategy no PAYMENT_STRATEGY_MAP: nenhuma linha de código existente precisou mudar.

5.2 Extensão 2 – Notificação WhatsApp

Arquivo adicionado: src/observers/whatsapp_observer.py

Arquivos modificados: nenhum

```
from src.observers.notification_observer import INotificationObserver
```

```
class WhatsAppObserver(INotificationObserver):
    def notify(self, cliente: str, tipo_cliente: str, evento: str) -> None:
        print(f"WhatsApp enviado para {cliente}: {evento}")
```

Por que foi simples: NotificationPublisher mantém uma lista de INotificationObserver. Basta instanciar WhatsAppObserver e chamar publisher.subscribe(WhatsAppObserver()): o publisher não sabe qual observer está registrando.

5.3 Extensão 3 – Desconto Progressivo por Volume

Arquivo adicionado: src/strategies/volume_discount.py

Arquivos modificados: nenhum

```
from src.strategies.discount_strategy import ItemDiscountStrategy
```

```
class VolumeDiscountStrategy(ItemDiscountStrategy):
```

```
    THRESHOLD = 3
```

```
    DISCOUNT = 0.15
```

```
    def __init__(self, base: ItemDiscountStrategy) -> None:
```

```
        self._base = base
```

```
    def apply(self, preco: float, quantidade: int) -> float:
```

```
        total = self._base.apply(preco, quantidade)
```

```
        if quantidade >= self.THRESHOLD:
```

```
            total *= (1 - self.DISCOUNT)
```

```
        return total
```

Por que foi simples: qualquer ItemDiscountStrategy pode ser decorada por VolumeDiscountStrategy. O OrderService usa o mesmo

ITEM_DISCOUNT_REGISTRY: basta registrar a estratégia decorada para o tipo desejado.

6 SAÍDA DOS TESTES E RELATÓRIO DE MÉTRICAS

Executado em 25/05/2026 — Python 3.12.6, Windows 11.

6.1 Testes Automatizados

```
$ py -m pytest --tb=no -q

..[ 85%]

..[100%]

84 passed in 0.41s
```

Resultado: 84/84 testes passando.

6.2 Cobertura de Código

```
$ py -m pytest --cov=src --cov-report=term-missing --tb=no -q
```

Name	Stmts	Miss	Cover
src\models\order.py	15	0	100%
src\models\order_item.py	14	0	100%
src\observers\notification_observer.py	30	1	97%
src\observers\whatsapp_observer.py	4	0	100%
src\repositories\order_repository.py	36	0	100%
src\services\notification_service.py	12	0	100%
src\services\order_service.py	51	1	98%
src\services\payment_service.py	21	0	100%
src\services\report_service.py	27	0	100%
src\services\stock_service.py	12	0	100%
src\strategies\crypto_payment.py	12	0	100%
src\strategies\discount_strategy.py	26	0	100%
src\strategies\payment_strategy.py	28	0	100%
src\strategies\volume_discount.py	11	0	100%
TOTAL	338	35	90%

Cobertura total: 90% (critério: $\geq 85\%$).

As 35 linhas não cobertas pertencem exclusivamente a src/main.py, que é ponto de entrada e não faz parte dos testes unitários.

6.3 Lint — PEP 8 / Boas Práticas (ruff)

```
$ py -m ruff check src/
All checks passed!
```

Resultado: 0 violações de estilo.

6.4 Checagem de Tipos Estáticos (mypy --strict)

```
$ py -m mypy --strict src/
Success: no issues found in 21 source files
```

Resultado: 0 erros de tipagem em 21 arquivos.

6.5 Complexidade Ciclomática (radon cc)

```
$ py -m radon cc src/ -s -a
src\services\payment_service.py
    M PaymentService.process - B (6)    <- pior caso
111 blocks analyzed.
Average complexity: A (1.75)
```

Complexidade média: A (1.75). Pior caso isolado: PaymentService.process com CC = 6 (grau B, limite do critério é CC ≤ 10).

6.6 Resumo das Métricas

Métrica	Resultado	Critério	Status
Testes passando	84 / 84	todos	OK
Cobertura de testes	90%	≥ 85%	OK
Lint / PEP 8 (ruff)	0 erros	0 erros	OK
Type hints (mypy)	0 erros em 21 arquivos	sem erros	OK
Complexidade ciclomática	média A (1.75), max B (6)	max CC ≤ 10	OK

Todos os critérios de qualidade foram atendidos.

7 REFLEXÃO METACOGNITIVA

7.1 O princípio que ainda não dominei

O princípio que ainda gera mais dúvida é o LSP (Liskov Substitution Principle). Os outros quatro têm critérios mais objetivos: SRP se avalia pela coesão, OCP pelo uso de extensão sem modificação, ISP pelo tamanho das interfaces, DIP pela direção das setas de dependência.

LSP é mais sutil porque a violação nem sempre gera erro: como em `PedEspecial.upd_st`, que ignora silenciosamente notificações e pontos. O código compila, os testes básicos passam, mas o comportamento do subtipo diverge do supertipo de forma observável apenas em cenários específicos.

A dúvida, como distinguir uma violação de LSP de uma extensão legítima de comportamento? Quando um override está "expandindo" o contrato do pai versus "quebrando" o contrato? O entendimento atual é que qualquer override que remova efeitos colaterais que o chamador espera do pai é uma violação: mas reconhecer isso no dia a dia ainda requer esforço.

7.2 Como usei IA neste trabalho

A IA funcionou como um auxiliar técnico ao longo dos três sprints, útil em pontos específicos, mas exigindo direcionamento constante.

No **Sprint 0**, a IA ajudou a mapear quais fluxos do legado mereciam teste antes de escrever qualquer assert, funcionou como uma segunda opinião para identificar os cenários (VIP, corporativo, boleto, cancelamento). A estrutura dos fixtures com `tmp_path` e `monkeypatch` também veio de uma sugestão que fez sentido e foi mantida sem alteração.

No **Sprint 1**, a discussão foi sobre quais métodos pertencem à `IOrderRepository` sem violar ISP. A primeira sugestão incluía métodos de relatório dentro do repositório (`get_sales_report`), misturando responsabilidades. A decisão foi separar, enquanto o repositório faz apenas CRUD, o relatório fica no service.

No **Sprint 2**, a IA foi eficiente para resolver a cadeia de erros de `tuple[object, ...]` no `OrderRepository` com `mypy --strict`, e ajudou a estruturar o padrão

Observer/Pub-Sub, sugerindo a separação entre INotificationObserver, NotificationPublisher e build_publisher. Em contrapartida, em um momento sugeriu implementar as extensões OCP adicionando novas classes *dentro* dos arquivos já existentes (payment_strategy.py, discount_strategy.py), o que seria modificação dos originais, sendo descartada a sugestão.

Decisão de design em que discordamos da IA: para VolumeDiscountStrategy, a IA propôs um decorator pattern mais elaborado, com sobrecarga de operadores e configuração por dicionário. Optamos pela versão mais simples, composição direta com _base: ItemDiscountStrategy e um único método apply.

O que não funcionou bem foram perguntas abertas de arquitetura. as respostas tendiam a complicar demais. Perguntas fechadas ("essa interface respeita ISP?", "esse construtor viola DIP?") foram muito mais produtivas.